VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
INTERNATIONAL UNIVERSITY

Report Project

Plants vs Zombies

INSTRUCTOR: Tran Thanh Tung, Thai Trung Tin

Course: Object-Oriented Programming_S1_2025-26_G01

Table of content

1. Introduction

Plan vs Zombies is a tower defense gameplay in which players use plants to defend against waves of zombies.

The primary goal of this project was to create a functional clone of the *Plants vs Zombies* (PVZ) style game. This exercise aimed to apply principles of Object-Oriented Programming (OOP) and develop a robust, modular design structure suitable for game development.

1. **Technology Stack:** The game was built using Java, utilizing the LibGDX framework. LibGDX was chosen for its cross-platform compatibility and extensive tools for rendering, input handling, and game loop management, aligning well with the need for modular, high-performance updates and rendering typical of game development.
2. **Scope:** The project successfully implemented core gameplay elements including the main `GameScreen`, a transparent `GameOverScreen` overlay, various plant and zombie entities, projectile mechanics (Pea/FrozenPea), a sun collection/cost system, the Heads-Up Display (HUD), structured zombie waves across defined lanes, and the single-use lawn mower defense. Critical elements like collision detection, game state updates, and rendering loops were fully integrated.

---

2. Game Overview + Image Placeholders

The game is played on a fixed grid where players strategically place plants to defend against incoming waves of zombies. The core mechanics involve resource management and tactical placement.

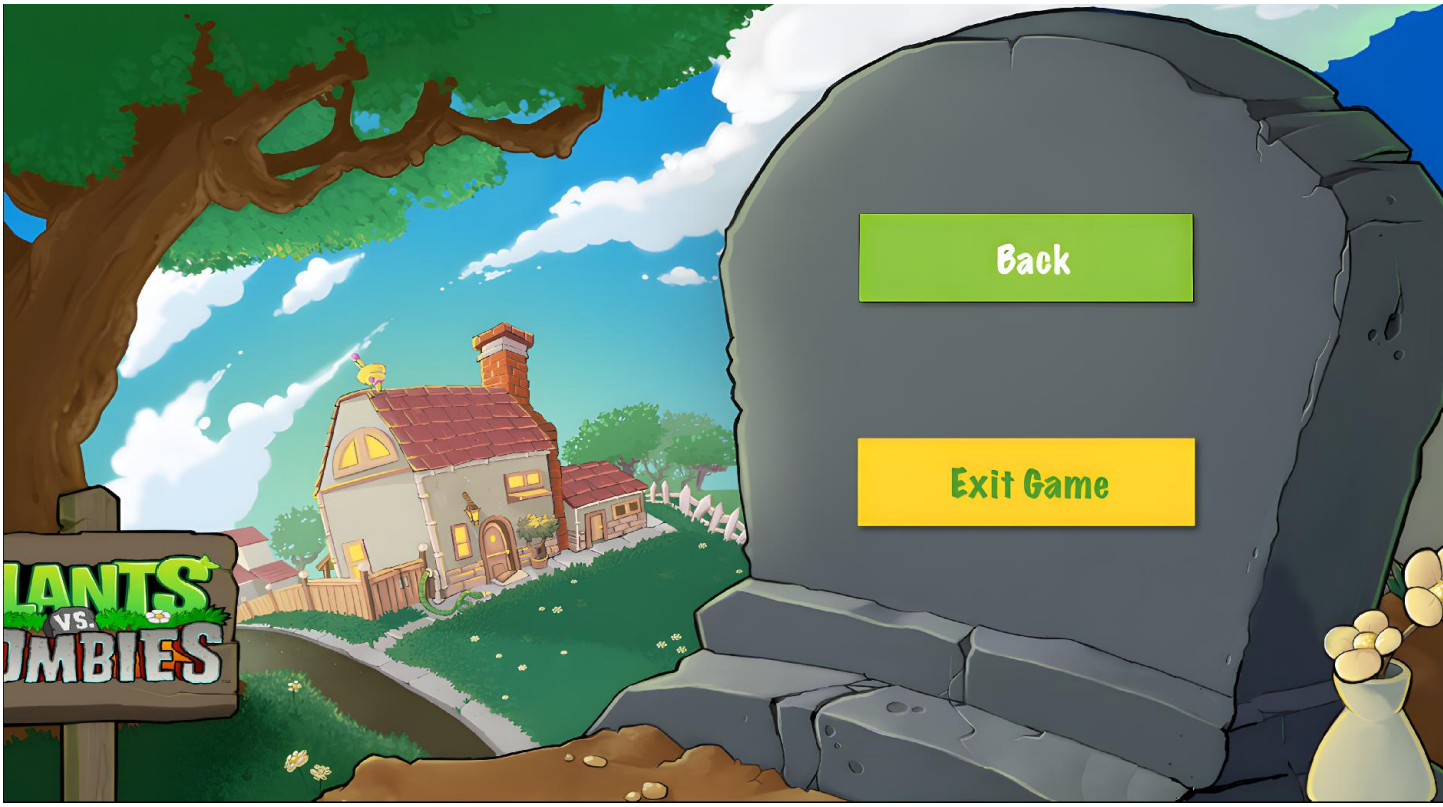
- **Grid Placement:** Plants are placed onto the grid, with each occupying a single cell. Placement requires spending "sun," the primary in-game currency.
- **Sun System:** Sun resources drop randomly on the screen and must be collected by the player to be added to their total. Certain plants (e.g., Sunflower) generate sun periodically.

- **Waves and Lanes:** Zombies spawn in timed waves and travel along fixed horizontal lanes towards the player's house.
- **Projectiles and Collision:** Plants attack using projectiles (e.g., `PeaProjectile`, `FrozenPeaProjectile`), which travel across the lane and deal damage upon collision with a zombie. Collision checks are performed during the update loop.
- **Game Over Condition:** The game ends when a zombie successfully breaches the last column of the grid and reaches the player's house, or if the special defense item (Lawn Mower) for that lane has already been consumed. (To be completed by the team).

Gameplay

Sun Production System

Game Over Screen



Resume Screen

3. Architecture

3.1 Packages

The project utilizes an Entity-Component-System (ECS)-inspired approach to decouple data (Components) from logic (Systems). Entities are simple containers holding various Components, while Systems operate on Entities that possess specific Component sets.

Package	Key Classes	Responsibilities
`pvz.com.screens`	`GameScreen`, `GameOverScreen`	Manage the display and control flow for different stages of the game (main game loop vs. game end).
`pvz.com.logic`	`GameWorld`, `GameState`, `ZombieWaveController`, `HudController`, `LawnMowerController`	High-level game coordination, state management, timing, and control of non-entity game elements.
`pvz.com.entities`	`Entity`, `Plant`, `Zombies`, `Sun`, `Projectile`, `PeaProjectile`, `FrozenPeaProjectile`	Base classes for all game objects and their hierarchical specializations.
`pvz.com.entities.components`	`Position`, `Bounds`, `Health`, `Animation`, `PlantAttack`, `Cooldown`, `Damage`, `Effect`, `GridCell`, `Cost`, `State`	Data containers defining an Entity's attributes and state.
`pvz.com.systems`	`Render`, `Animation`, `Movement`, `Collision`, `Cleanup`, `PlantAttack`, `SunProduction`, `SunPickup`	Core logic modules that process Entities based on their Components.

Interfaces:

- **`IGameSpawner`:** Interface used by controllers (e.g., `ZombieWaveController`) to abstract the creation and placement of new entities into the `GameWorld`.
- **`ISunReceiver`:** Interface implemented by classes that need to process collected sun resources (e.g., `HudController` or `GameState`).

3.2 Responsibilities

3.2.1 Plant class

a. Types of plants

In this project, plants act as stationary defensive entities. Each plant type differs in attack behavior, cooldown, and HP.

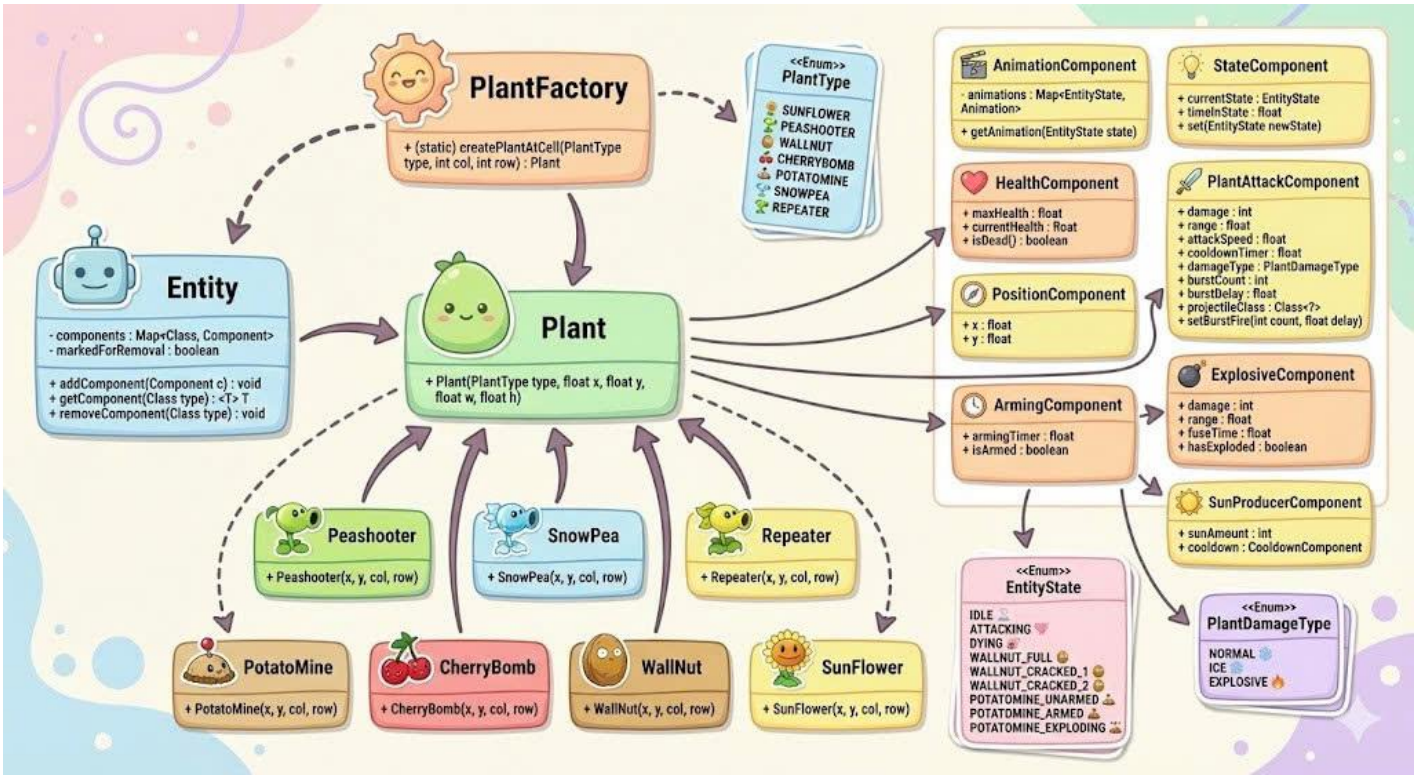
Common plant types include:

- **PeaShooter** – Basic ranged attacker
  Cost: 100 sun · *Cooldown:* 2.5s · *Hp: 100*
- **Sunflower** – Generates sun resources
  Cost: 50 sun · *Cooldown:* 2.5s · *Hp: 100*
- **Wall-nut** – High-HP defensive plant
  Cost: 50 sun · *Cooldown:* 10s · HP: 4000
- **Cherry Bomb** – Area-of-effect explosive damage
  Cost: 150 sun · *Cooldown:* 15s · *Hp: no need hp because this will disappear once exploded)*
- **Potato Mine** – Delayed explosive trap
  Cost: 25 sun · *Cooldown:* 10s · *Hp: (ungrow):, grow: no hp*
- **Repeater** – Fires multiple projectiles
  Cost: 200 sun · *Cooldown:* 2.5s · *Hp: 100*
- **Snow Pea** – Ranged attacker with slow effect
  Cost: 175 sun · *Cooldown:* 2.5s · *Hp: 100*

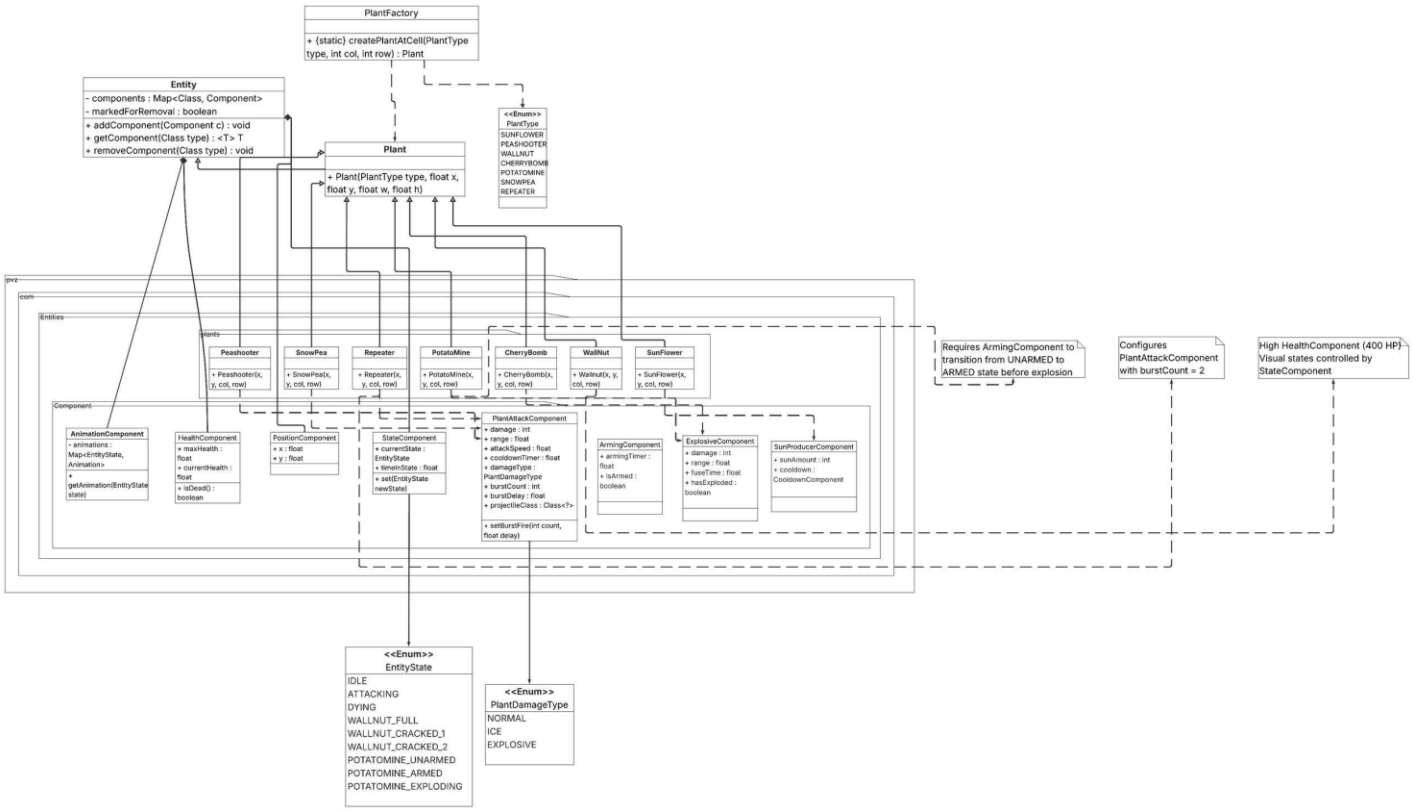
All plants share the same base structure and differ mainly through their components and configuration values.

Plants

b. Base Plant Structure



Minimal Base Plant Structure



Full Base Plant Structure

All plants inherit from an abstract `Plant` class, which defines shared properties such as grid position and basic lifecycle states.

```
public abstract class Plant extends Entity {

    protected int row;
    protected int col;

    protected boolean dead = false;
    protected boolean isDying = false;

    public Plant(int row, int col) {
        this.row = row;
        this.col = col;
    }
}
```

This base class allows different plant types to be treated uniformly by Systems.

c. Attack Mechanics

Attacking plants use an `AttackComponent` and `CooldownComponent` to control firing behavior.

- The `PlantAttackSystem` checks whether a plant is ready to attack.
- If the cooldown has expired, the plant spawns a projectile entity.
- Damage is applied through collision with zombies.

This design decouples attack logic from plant classes and keeps behavior data-driven.

d. Resource Generation (Sunflower)

Plants that generate resources do not attack.
Instead, they use a production cooldown:

- A `SunProductionComponent` tracks generation intervals.
- When ready, sun is spawned and added to the player's resource pool.
- No direct interaction with zombies is required.

e. Health and Damage Handling

Plants receive damage from zombies when being attacked.

```
@Override
public void takeDamage(int damage) {
    if (dead || isDying)
        return;

    health -= damage;

    if (health <= 0)
        startDeath(false);
}
```

The damage-handling logic is consistent with zombies, simplifying system behavior and debugging.

f. Death and Removal

When a plant's HP reaches zero:

- The plant enters a dying state.
- Death animation is played.
- The plant is marked for removal after the animation finishes.
- The grid cell becomes available for new plants.

This ensures correct synchronization between visuals, gameplay logic, and grid management.

g. ECS Design Rationale

The `Plant` class itself contains minimal logic.
Most behavior is defined by:

- Components (Attack, Health, Cooldown, Production)
- Systems (PlantAttackSystem, CollisionSystem, ResourceSystem)

```
public class Plant extends Entity {

    public Plant(float x, float y, float width, float
height) {
        super();

        this.addComponent(new PositionComponent(x, y));
        this.addComponent(new SizeComponent(width,
height));
        this.addComponent(new BoundsComponent(x, y, width,
height));
    }
}
```

This approach:

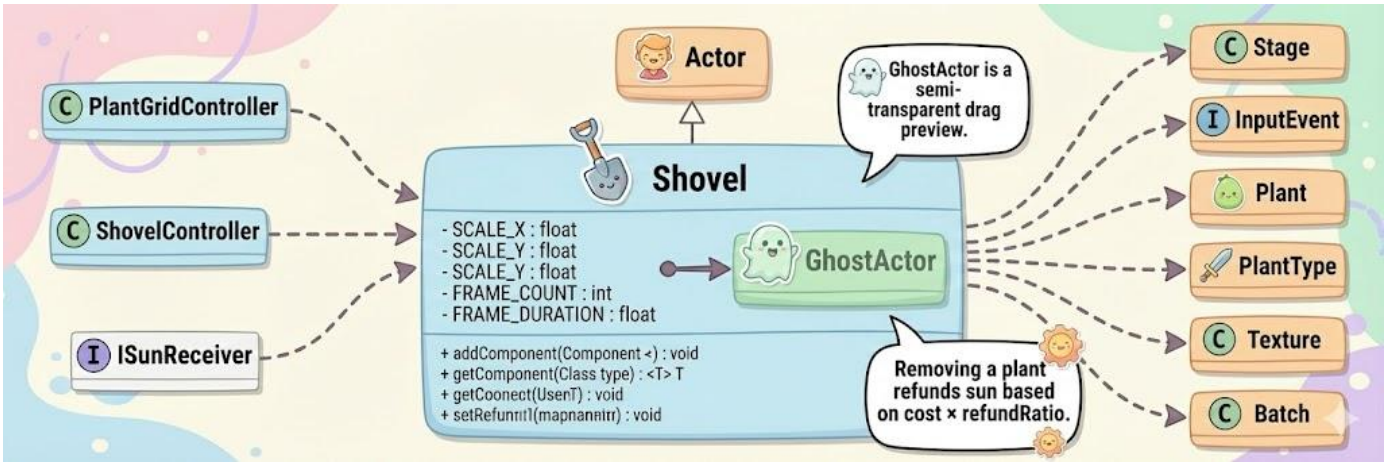
- Reduces inheritance complexity
- Makes it easy to add new plant types
- Keeps the gameplay loop clean and scalable

h. Plant Removal and Sun Refund (Shovel)

To support flexible gameplay, the project includes a **Shovel** tool that allows the player to remove an already-placed plant from the grid. This feature is implemented as a **Scene2D HUD Actor** (`Shovel extends Actor`) and works independently from plant combat logic, keeping the ECS gameplay loop clean.

Interaction flow

- The shovel icon is displayed on the HUD.
- When the player **left-clicks and drags**, the shovel becomes active and shows a semi-transparent **ghost preview** that follows the cursor.
- On **release**, the shovel attempts to remove the plant at the nearest grid cell under the ghost position.



Shovel - UML Chart

Grid targeting

- The shovel uses the HUD `Stage` to convert the ghost's center position into **screen coordinates**, then calls `grid.screenToNearestCell(...)` to locate the closest `(row, col)` cell.
- If there is a plant in that cell (`grid.getPlantAt(row, col)`), removal proceeds; otherwise the action is ignored.

Refund mechanism

- Before removal, the shovel reads the plant's type through ECS:

```
private PlantType getPlantType(Plant plant) {

    PlantTypeComponent tc =
plant.getComponent(PlantTypeComponent.class);

    return (tc != null) ? tc.type : null;

}
```

- If a valid `PlantType` exists, the shovel queries `PlantCatalog` to get the plant's original **cost**, then computes a refund using a configurable ratio:
  - `refund = round(cost * refundRatio)`
  - `refundRatio` is controlled by `DesignConfig.SHOVEL_REFUND_RATIO` (and can be changed via `setRefundRatio()`).
- The refund is returned to the player via `ISunReceiver.addSun(refund)`.

Removal execution

- After refunding (if applicable), the actual removal is delegated to `ShovelController.tryRemovePlant(row, col)`, which keeps UI input logic (HUD) separate from grid state updates (game logic).

UI scaling and cleanup

- `layoutTopLeft(hudWorldWidth, hudWorldHeight)` scales the shovel icon consistently across resolutions using `ScaleManager`, and places it at the top-left HUD area.

- Textures used by the shovel (icon + ghost) are released in `dispose()` to prevent memory leaks.

This design keeps the shovel as a **HUD-level tool** (input + visuals), while plant identity and economy rules remain **data-driven** via components (`PlantTypeComponent`) and catalogs (`PlantCatalog`), aligning well with the project's ECS rationale.

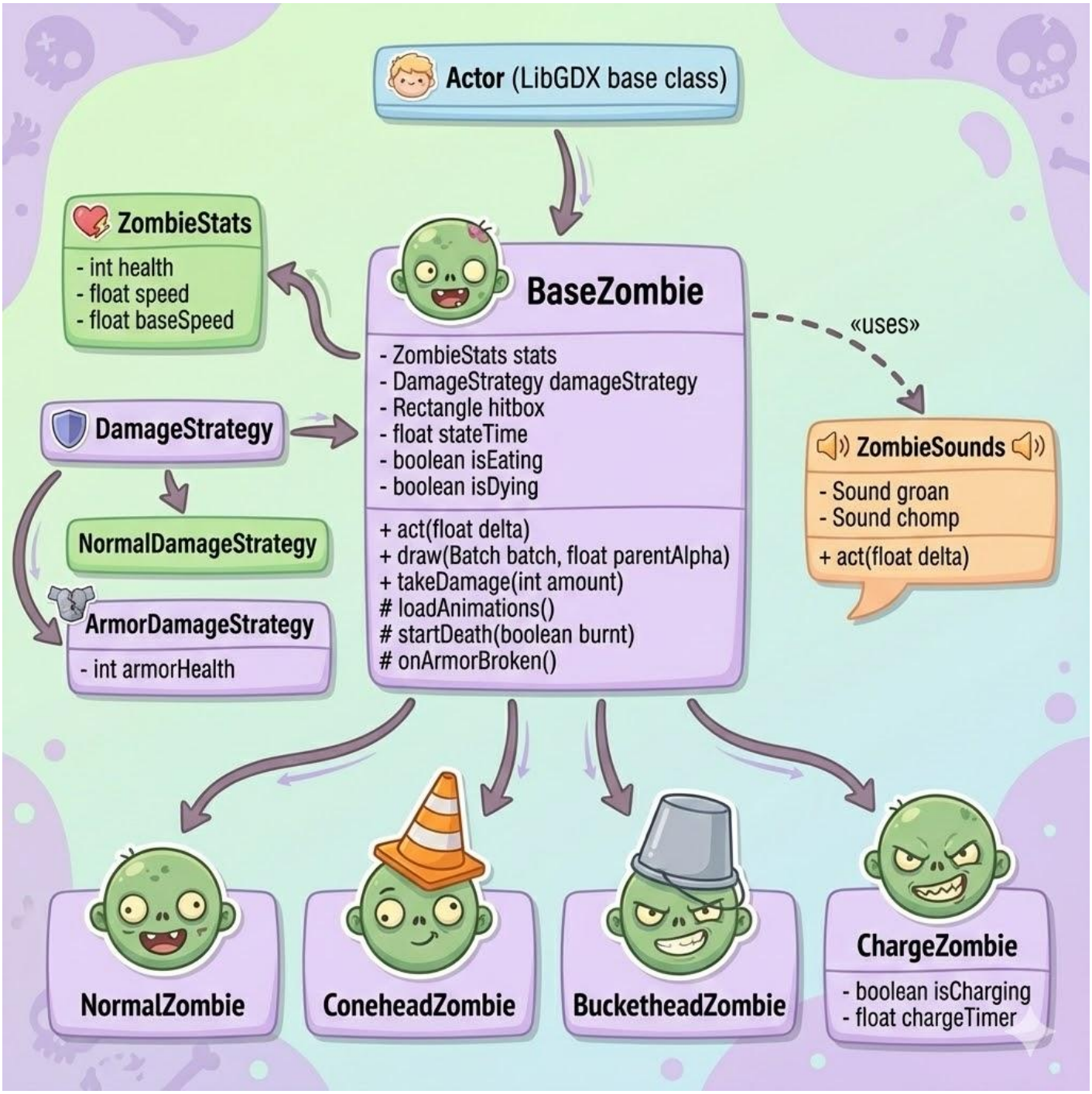
3.2.2 Zombie class

Zombies Illustration

a. Types of zombies

There are 4 types of zombies in our project:

- **Normal zombie:** Hp :100; speed:15f
- **Conehead zombie:** Hp:100; Conehead:200;speed:18f
- **Buckethead zombie:** Hp:100; Buckethead:300;speed:18f
- **Charge zombie:** Hp: 160; speed: 32f



Zombie Packages - Flow Chart

b. HP Mechanics

The HP of zombie is counted by:

```
@Override
public void takeDamage(int damage) {
     if (dead || isDying)
          return;
     health -= damage;
     if (health <= 0)
          startDeath(false);
}
```

This method ensures that:

- Zombies do not receive damage multiple times after entering the dying state.
- Damage is subtracted directly from the `health` attribute.
- When HP reaches zero or below, the zombie transitions into the death process.
- This method directly manages zombie damage and ensures the integrity of the death process. Specifically:
- Damage is subtracted directly from the zombie's health.
- The zombie initiates the death sequence immediately upon its HP dropping to zero or less.
- This prevents a zombie from sustaining additional, unnecessary damage after it has already entered the dying state.

c. Movement Behavior

Each zombie type has a predefined movement speed, which is stored in its corresponding component (e.g., `MovementComponent`).

- Normal Zombie: slow, constant movement
- Conehead / Buckethead Zombie: same movement speed as Normal Zombie
- Charge Zombie: significantly higher speed

```
public class MovementSystem {

    public void update(List<Entity> entities, float
deltaTime) {
        for (Entity e : entities) {
```

```
            // bỏ qua entity đã bị đánh dấu xóa
            if (e.markedForRemoval)
                continue;
            PositionComponent pos =
e.getComponent(PositionComponent.class);
            MovementComponent mov =
e.getComponent(MovementComponent.class);
            if (pos != null && mov != null) {
                pos.x += mov.velocity.x * deltaTime;
                pos.y += mov.velocity.y * deltaTime;
            }
        }
    }
}
```

Movement is updated every frame by the `MovementSystem`, which adjusts the zombie's position based on its speed and delta time.

d. Special Attributes

Some zombie types have additional defensive properties:

- Conehead Zombie
  Has extra HP provided by the cone, allowing it to survive more attacks before dying.
- Buckethead Zombie
  Has the highest defensive HP among standard zombies, acting as a tank unit.
- Charge Zombie
  Trades durability for speed, allowing it to reach plants faster and apply pressure on the player.

These attributes are implemented through different initial values of HP and speed rather than separate logic branches, keeping the system flexible and extensible.

e. Death Handling

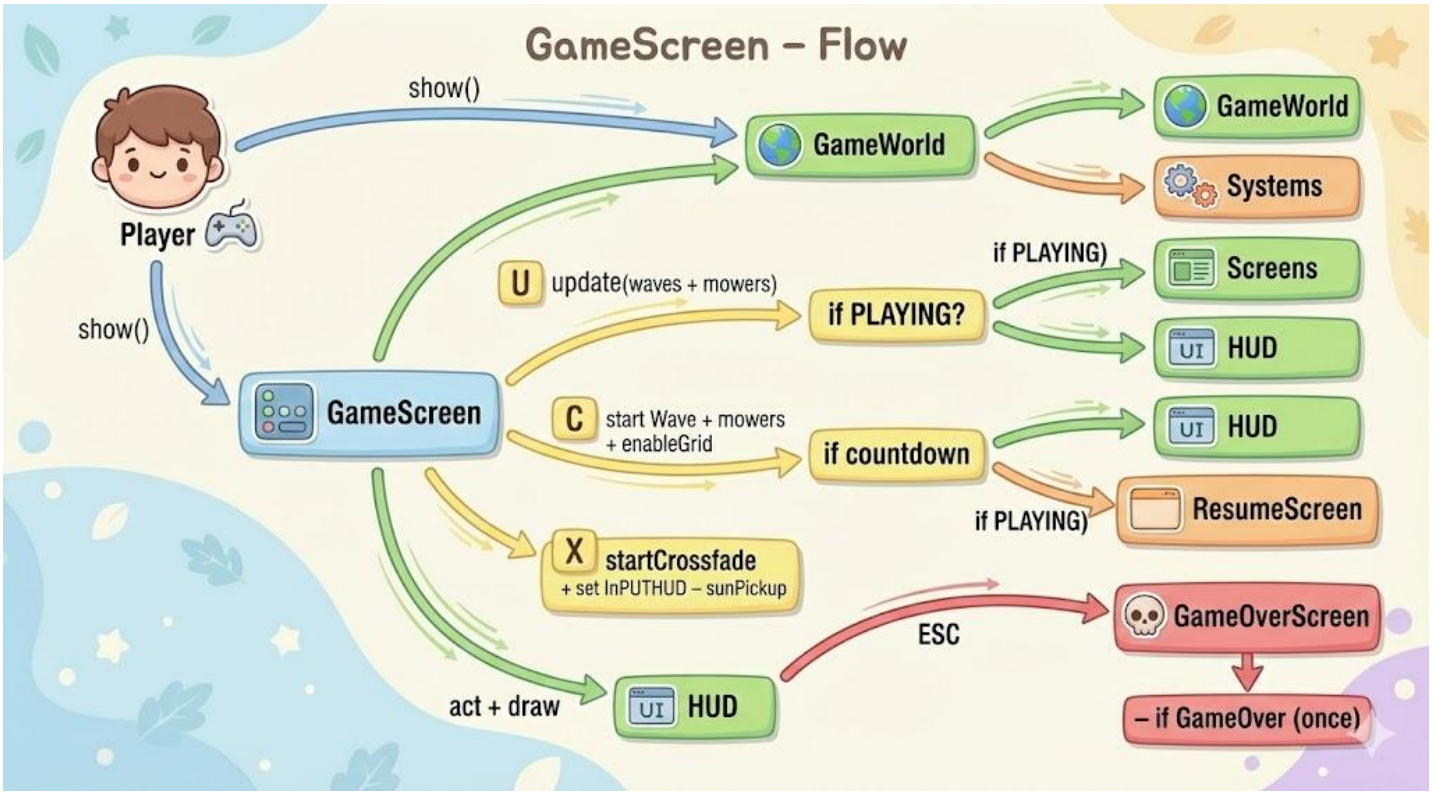
When a zombie's HP reaches zero:

- The zombie enters a dying state.
- Death animation is played.
- The entity is marked for removal after the animation finishes.
- Systems will remove the zombie safely from the game world.

This approach avoids immediate deletion and prevents update conflicts during the same frame.

---

4. UML (Placeholders)



Play Process

Key Architectural Dependencies:

- The `GameScreen` manages the `GameWorld`, driving the update/render loop.
- The `GameWorld` is responsible for initializing and coordinating all `Systems`.
- High-level logic controllers (e.g., `ZombieWaveController`) depend on the `IGameSpawner` interface to inject new `Entities` into the `GameWorld`.

- `Systems` depend only on `Entities` and `Components`, ensuring logic is separated from high-level game flow.
- New entities (e.g., `PeaProjectile`) are composed of multiple components (e.g., `Position`, `Bounds`, `Damage`).
- The `CollisionSystem` checks interactions between all `Entities` possessing both `Position` and `Bounds` components.

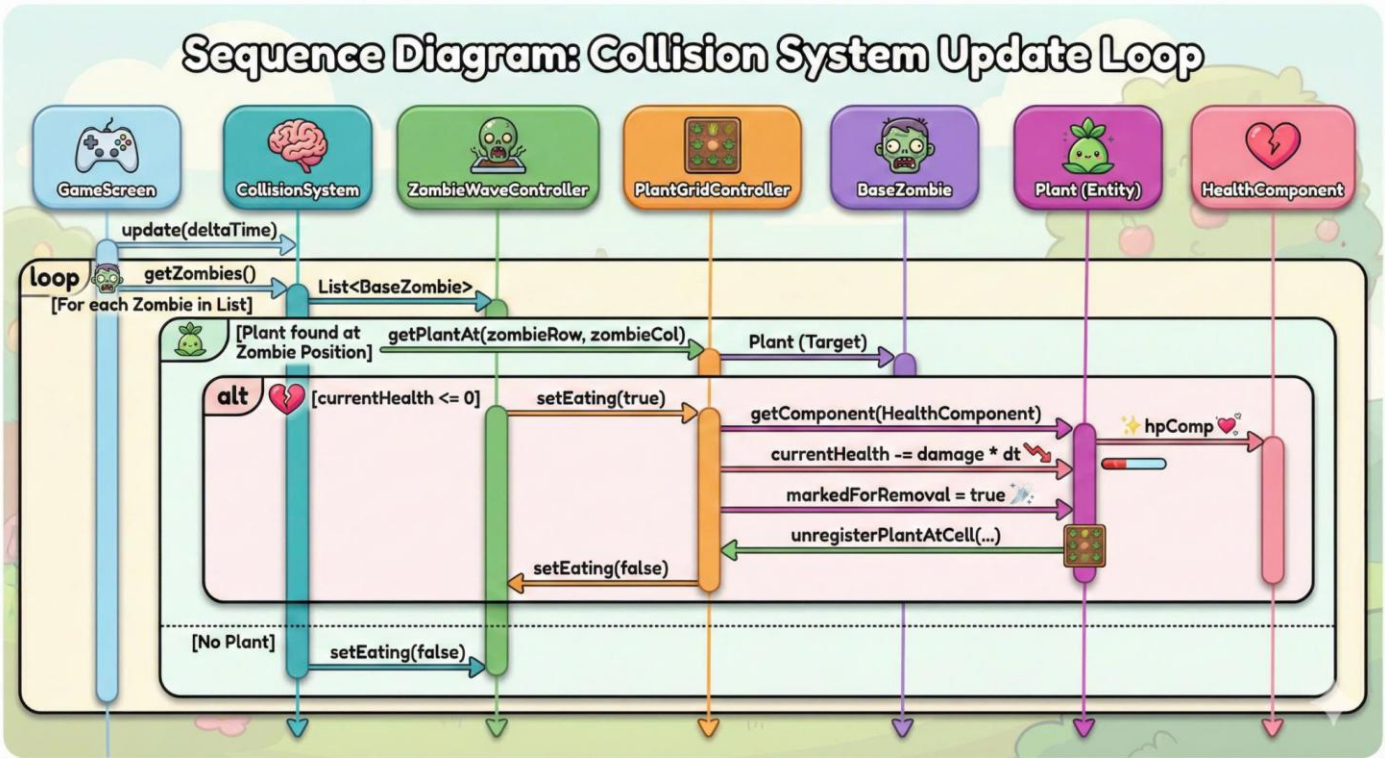
---

5. SOLID Principles

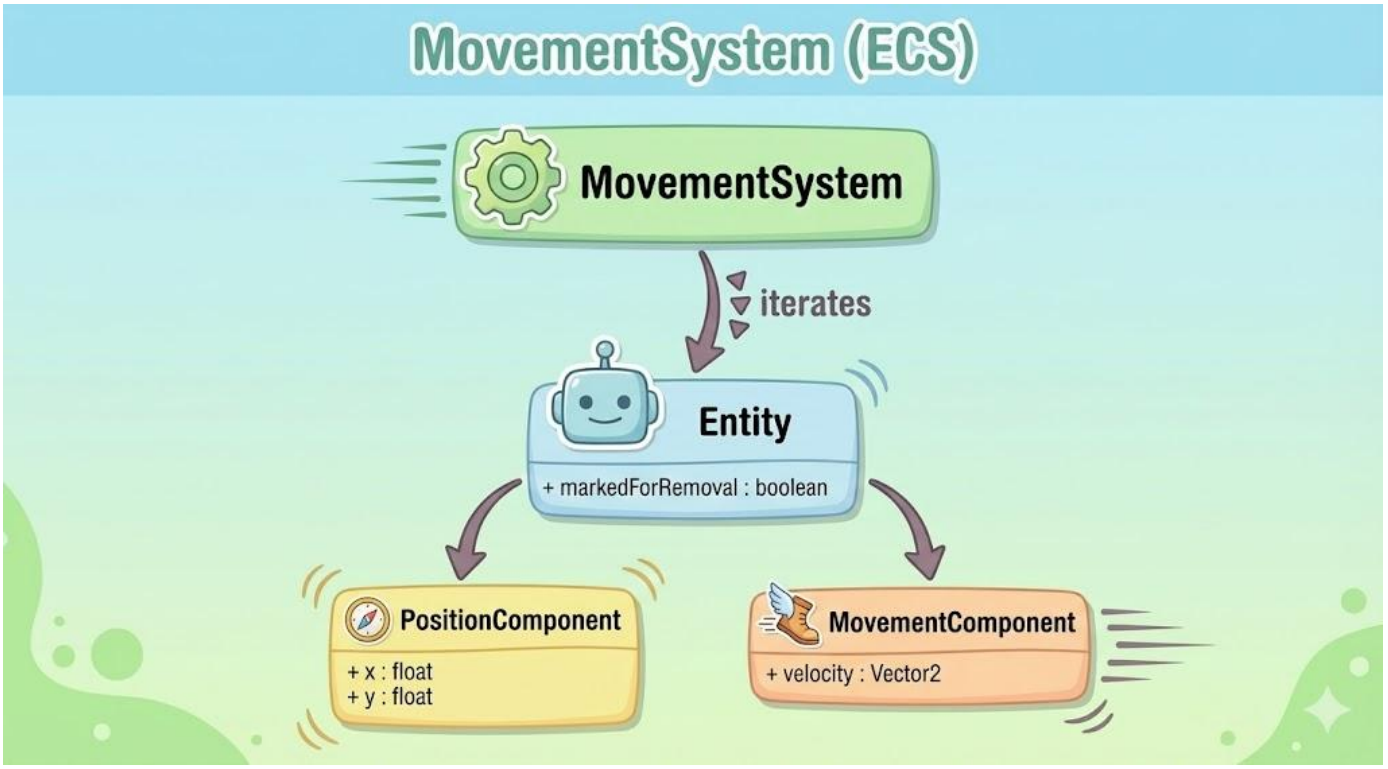
The design of the PVZ clone adheres to the SOLID principles to ensure maintainability, flexibility, and scalability.

Single Responsibility Principle (SRP)

- **Explanation:** Each class or system should have only one reason to change.
- **Evidence:** Each `System` (e.g., `MovementSystem`, `CollisionSystem`) handles a single, distinct aspect of the game logic, operating only on the relevant components.



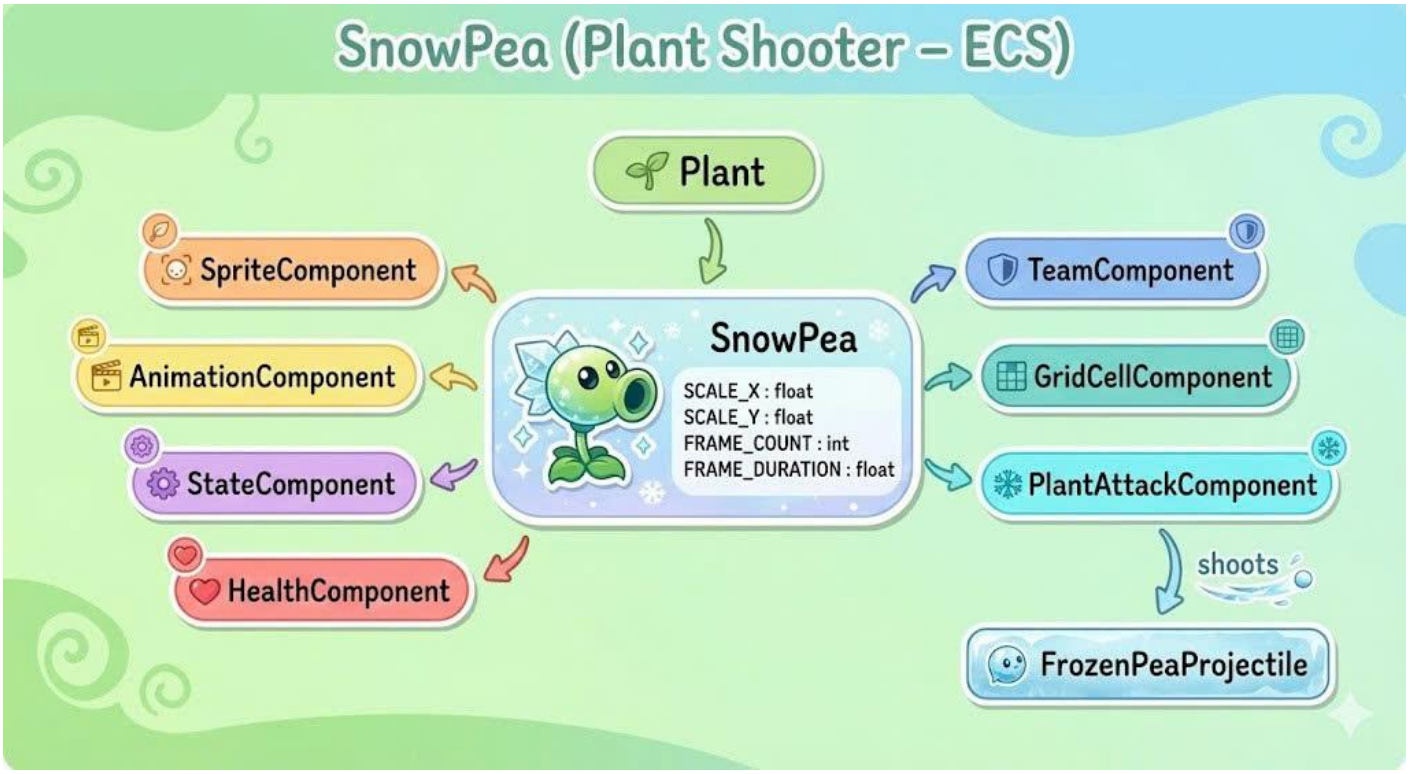
Collision System



Movement System

Open/Closed Principle (OCP)

- **Explanation:** Software entities should be open for extension but closed for modification.
- **Evidence:** Adding a new plant type (e.g., a Freezing Sunflower) only requires creating a new `Plant` subclass and combining existing or new `Components`, without modifying existing core systems (e.g., `RenderSystem`, `PlantAttackSystem`).



Snow Pea - Flow Chart

Liskov Substitution Principle (LSP)

- **Explanation:** Subtypes must be substitutable for their base types without altering the correctness of the program.
- **Evidence:** All subclasses of `Plant` or `Zombie` can be treated generically as their base types within the `Systems` loop, as their specialized behaviors are defined via their Components, not by overriding fundamental base class behavior that could cause errors.
- [PLACEHOLDER: Code excerpt demonstrating substitution of a subclass for a base class in a loop]

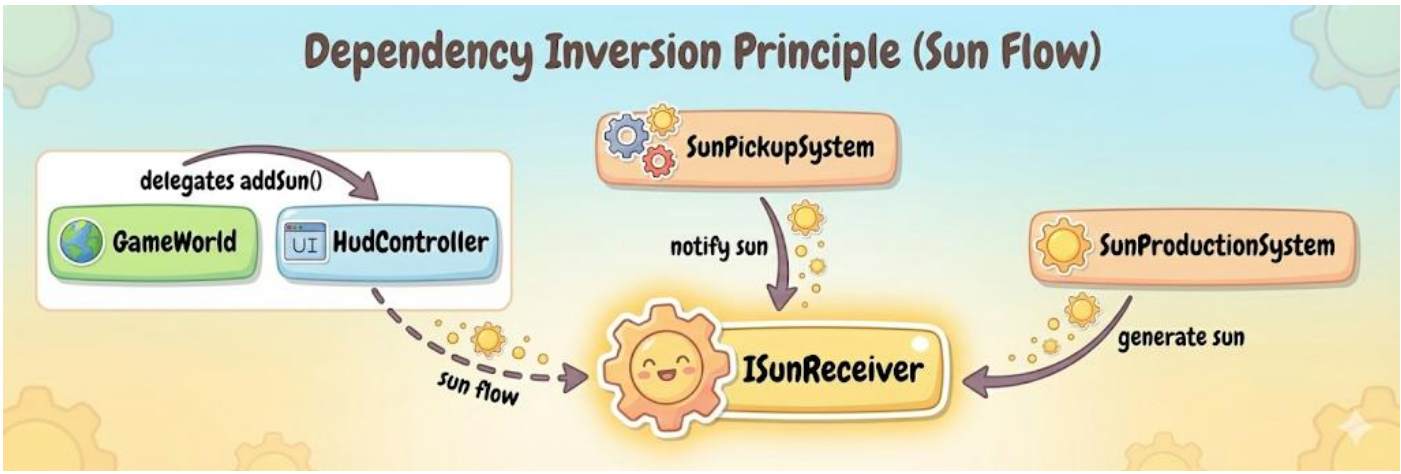
Interface Segregation Principle (ISP)

- **Explanation:** Clients should not be forced to depend on interfaces they do not use.
- **Evidence:** The use of small, specific interfaces like `IGameSpawner` (only for creating entities) and `ISunReceiver` (only for receiving sun events) ensures classes only implement the methods they require.

```
public interface ISunReceiver {
    void addSun(int amount);
}
```

Dependency Inversion Principle (DIP)

- **Explanation:** High-level modules should not depend on low-level modules; both should depend on abstractions.
- **Evidence:** The `GameWorld` (high-level) coordinates the `Systems` (low-level logic) via interfaces or base classes, rather than relying on concrete implementations. The `HudController` depends on the `ISunReceiver` abstraction.



Dependency Inversion Principle

Summary of SOLID Application

Principle	Where Applied	Benefit
SRP	Entity Systems	Clear separation of concerns; easier maintenance.

Principle	Where Applied	Benefit
OCP	Plant/Zombie subclasses	Simplified extension; adding new content is faster.
LSP	Entity handling in Systems	Safe polymorphism; reliable entity processing.
ISP	`IGameSpawner`, `ISunReceiver`	Decoupled client logic; reduced interface bloat.
DIP	`GameWorld` coordination	Flexible architecture; easy to swap out system implementations.

6. Design Pattern

6.1. Strategy Pattern

- **Application:** Used to handle the damage calculation logic for different Zombie types (e.g., normal health vs. armored protection).
- **Evidence:**
  - The **Zombie Packages Flow Chart** illustrates a DamageStrategy interface with concrete implementations: NormalDamageStrategy and ArmorDamageStrategy.
  - The BaseZombie class delegates damage handling to a damageStrategy object instead of implementing the logic directly.
- **Benefit:** Allows the game to support new zombie defense mechanisms (e.g., Buckethead, Conehead) without modifying the core BaseZombie class, adhering to the **Open/Closed Principle**.

6.2. Factory Pattern

- **Application:** Centralizes the complex instantiation logic of Plant entities, which involves assembling multiple ECS components.
- **Evidence:**
  - The **Plant Structure Diagram** explicitly defines a PlantFactory class with a static method createPlantAtCell.
  - This factory is responsible for configuring specific components for plants, such as setting burstCount = 2 for the Repeater or attaching ArmingComponent for the Potato Mine.
- **Benefit:** Decouples the game logic (placing a plant) from the initialization details of specific plant types.

6.3. State Pattern

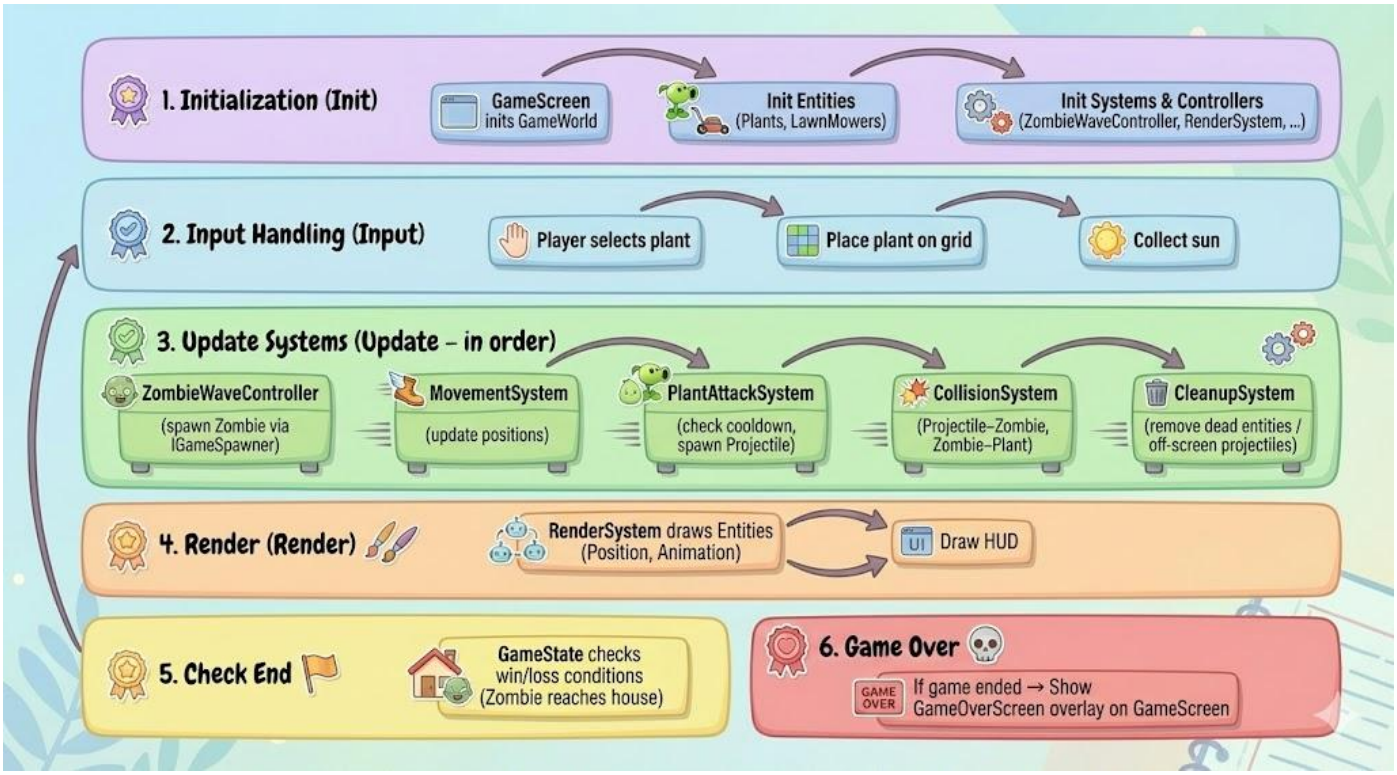
- **Application:** Manages the internal states of entities (e.g., idle, attacking, dying) and synchronizes them with visual animations.
- **Evidence:**
  - The **Plant Structure Diagram** features a StateComponent that tracks the currentState using an EntityState Enum.

    - States such as WALLNUT_CRACKED_1 and POTATOMINE_ARMED are defined to trigger specific behaviors and visual changes.
- **Benefit:** Eliminates complex conditional logic (nested if-else) in the update loops and ensures reliable state transitions

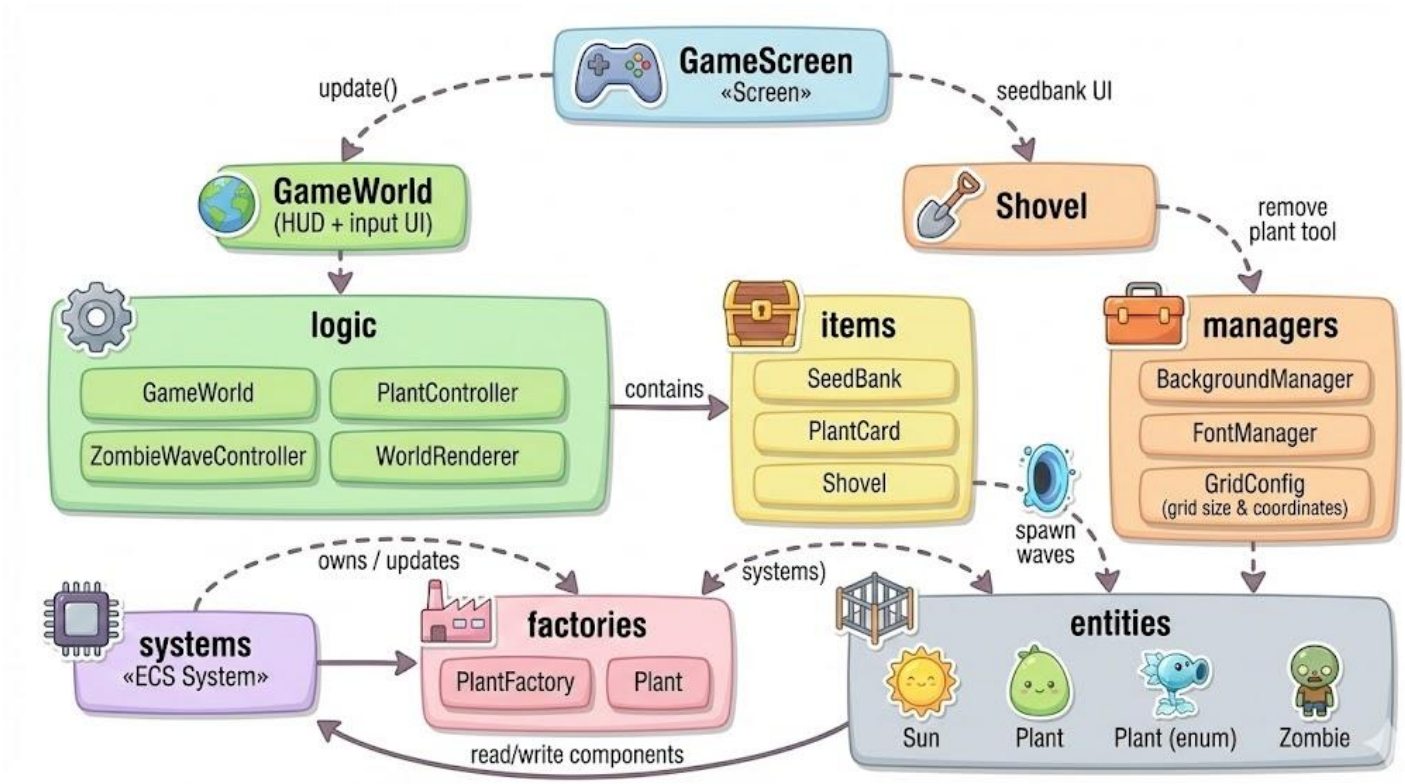
7. Game Flow

The game operates on a continuous loop, managed by the LibGDX framework, ensuring consistent updates and rendering.

- **Initialization (Init):** The `GameScreen` initializes the `GameWorld`, all `Entities` (pre-placed plants, lawn mowers), and all `Systems` and controllers (e.g., `ZombieWaveController`).
- **Input Handling (Input):** The game processes user input for plant selection, grid placement, and sun collection.
- **Update Systems (Update):**
    - `ZombieWaveController` updates, potentially spawning new `Zombies` via `IGameSpawner`.
    - `MovementSystem` updates positions of all moving entities.
    - `PlantAttackSystem` checks cool-downs and spawns `Projectiles`.
    - `CollisionSystem` detects hits (Projectile vs. Zombie, Zombie vs. Plant) and applies damage/effects.
    - `CleanupSystem` removes entities with zero health (dead Zombies/Plants) or out-of-bounds Projectiles.
- **Render (Render):** The `RenderSystem` draws all entities based on their `Position` and `Animation` components, followed by the HUD.
- **Check End:** `GameState` checks win/loss conditions (e.g., Zombie reaches the house).
- **Game Over:** If the game ends, the `GameOverScreen` overlay is rendered transparently over the main screen.



Game - Flow Chart



GameScreen - Flow Chart

---

8. Conclusion + Future Work

The *Plants vs Zombies (LibGDX)* project successfully demonstrates the application of OOP principles within a game development context, utilizing an ECS-inspired architecture for high modularity.

Results Achieved:

- A fully functional `GameScreen` with core PVZ gameplay loop.
- Implementation of the ECS pattern, clearly separating data (Components) and logic (Systems).
- Successful implementation of the sun economy, plant placement, and wave-based zombie spawning.
- Robust collision detection and entity cleanup mechanism.
- Adherence to all five SOLID principles, resulting in a maintainable codebase.

Future Work:

- **More Entities:** Introduce a wider variety of specialized Plants (e.g., Wall-nut, Cherry Bomb) and Zombies (e.g., Buckethead Zombie).
- **Balancing:** Extensive tuning of plant costs, zombie health, and wave difficulty.
- **Optimization:** Further performance optimization, particularly in the `CollisionSystem`.
- **UX/Sound:** Adding sound effects, background music, and a comprehensive main menu structure with level progression.

---

9. Links

SOURCES:

[1] https://libgdx.com/wiki/
[2] https://github.com/rohangoel96/PlantsVsZombies-Game
[3] https://github.com/GeeeekExplorer/PVZ

OUR GITHUB CODES:

[4] https://github.com/Huy-IU-2k6/Plant_vs_Zombie

Team Member

No.	Full Name	Student ID	Role	Main Responsibilities
1	Lê Thái Đức Tùng	ITDSIU24055	Design HUD/UI (Sun bar, cards, buttons)	Implement UI interactions/feedback (drag-drop, highlight)
2	Nguyễn Duy Phước	ITITWE23026	Build zombie types + stats + animations	Handle AI/state + collision + wave spawn
3	Phạm Gia Huy	ITCSIU24034	Build plant types + cost/cooldown + animations	Handle attack/production (projectiles/effects) + grid placement